Sprint 25 Structured Notes: Dynamic Programming

0. What Sprint 25 Is About

Sprint 25 is about **Dynamic Programming**, usually shortened to **DP**.

Dynamic programming is a problem-solving technique. It helps you make some recursive or repeated calculations faster by saving answers you already calculated.

Simple meaning:

> Dynamic programming means: do not solve the same smaller problem again if you already solved it once.

Before Sprint 25, you learned many JavaScript and algorithm skills:

- variables
- values
- data types
- strings
- numbers
- booleans
- conditions
- functions
- arrays
- loops
- objects
- callbacks
- higher-order array methods
- Sets and Maps
- classes
- Big O
- recursion
- searching
- sorting
- data structures
- graphs
- trees
- heaps

Sprint 25 uses many of those ideas together.

The main goal of Sprint 25 is:

Use stored answers to avoid repeated work in algorithm problems.

By the end of Sprint 25, you should understand:

- dynamic programming
- overlapping subproblems
- optimal substructure
- naive recursion
- why naive recursion can be slow
- memoization
- top-down dynamic programming
- tabulation
- bottom-up dynamic programming
- DP arrays
- memo objects
- space optimization
- climbing stairs
- coin change
- when to use dynamic programming
- time complexity of DP solutions
- space complexity of DP solutions
- common beginner DP mistakes

Part A: Current Progress Before Sprint 25

1. What You Already Know

Before Sprint 25, you already know enough JavaScript to understand DP.

Earlier Topic	What You Learned	How It Helps in Sprint 25
Variables	Store values with `let` and `const`	DP stores answers in variables, arrays, objects, or Maps.
Functions	Reuse logic with parameters and return values	DP problems are usually written as functions.
Conditions	Choose what code runs	DP uses base cases and checks.
Arrays	Store ordered values	Tabulation often uses a `dp` array.
Objects	Store key-value pairs	Memoization often uses an object as a cache.
Loops	Repeat work safely	Tabulation uses loops to fill a table.

Earlier Topic	What You Learned	How It Helps in Sprint 25
Recursion	A function calls itself	Memoization improves recursive solutions.
Big O	Measure time and space growth	DP is often used to improve time complexity.
Searching and Sorting	Solve algorithm problems	DP is another algorithmic problem-solving technique.

Simple summary:

> Earlier sprints taught you how to write JavaScript and solve algorithm problems. Sprint 25 teaches you how to avoid repeated work inside those solutions.

---

2. The Main Shift in Sprint 25

Before Sprint 25, you may solve a problem by recursion.

Example idea:

```
function solve(problem) {
  return solve(smallerProblemA) + solve(smallerProblemB);
}
```

This can work, but it can also repeat the same smaller problems many times.

Sprint 25 adds a new question:

> Am I calculating the same answer again and again?

If the answer is yes, dynamic programming may help.

Part B: The Core Idea of Dynamic Programming

3. What Is Dynamic Programming?

Dynamic programming is a technique for solving problems by:

1. Breaking a big problem into smaller problems.
2. Solving the smaller problems.

3. Saving the answers.
4. Reusing saved answers later.

Simple meaning:

> DP saves answers so the program does not repeat expensive work.

Example idea:

```
// If we already know the answer, return it.
if (memo[n] !== undefined) {
  return memo[n];
}
```

That small check can make a solution much faster.

---

4. Dynamic Programming Mental Model

Imagine you are doing homework.

A friend asks:

> What is 12 × 12?

You calculate:

```
12 × 12 = 144
```

Later, another friend asks the same thing.

You do not need to calculate it again. You already know the answer.

You just say:

```
144
```

Dynamic programming works similarly.

The program says:

> I already solved this smaller problem. I will reuse the answer.

5. Why Dynamic Programming Matters

Some recursive solutions look simple, but they can become very slow.

Why?

Because they repeat the same calculations.

For a small input, this may not matter.

For a large input, repeated work can make the program extremely slow.

Dynamic programming helps by saving repeated results.

Simple comparison:

Approach	Behavior
Naive recursion	Solves the same subproblems many times.
Dynamic programming	Solves each important subproblem once, then reuses it.

Part C: Two Conditions for Dynamic Programming

Dynamic programming is useful when a problem has two main features:

1. overlapping subproblems
2. optimal substructure

---

6. Overlapping Subproblems

Overlapping subproblems means:

> The same smaller problem appears more than once.

Example:

```
climbStairs(5)
```

To solve this, you may need:

```
climbStairs(4) + climbStairs(3)
```

But to solve `climbStairs(4)`, you need:

```
climbStairs(3) + climbStairs(2)
```

Now notice the repeat:

```
climbStairs(3)
```

It appears more than once.

That is an overlapping subproblem.

Simple meaning:

> If the same smaller question keeps coming back, it is overlapping.

---

7. Optimal Substructure

Optimal substructure means:

> The answer to the big problem can be built from answers to smaller problems.

Example:

```
climbStairs(n) = climbStairs(n - 1) + climbStairs(n - 2)
```

This means:

- To reach step `n`, you could come from step `n - 1`.
- Or you could come from step `n - 2`.
- So the answer for `n` depends on smaller answers.

Simple meaning:

> If smaller correct answers help build the bigger correct answer, the problem has optimal substructure.

---

8. Quick Test: Is This a DP Problem?

Ask these questions:

1. Can I break the problem into smaller versions of the same problem?
2. Do the same smaller problems repeat?
3. Can I build the final answer from smaller answers?
4. Would saving earlier answers make the solution faster?

If the answer is yes, dynamic programming may be useful.

Part D: Naive Recursion

9. What Is Naive Recursion?

Naive recursion is the simple recursive solution before optimization.

Simple meaning:

> Naive recursion solves the problem directly, but it does not save repeated answers.

Example structure:

```
function solve(n) {
  if (baseCase) {
    return baseAnswer;
  }

  return solve(n - 1) + solve(n - 2);
}
```

This may be easy to understand, but it can be slow.

---

10. Why Naive Recursion Can Be Slow

Naive recursion can be slow because it recalculates the same values.

Example:

```
climbStairs(5)
```

breaks into:

```
climbStairs(4) + climbStairs(3)
```

Then:

```
climbStairs(4)
```

breaks into:

```
climbStairs(3) + climbStairs(2)
```

Now `climbStairs(3)` is calculated more than once.

For bigger numbers, repeated calculations grow quickly.

Simple rule:

> Naive recursion is often easy to write, but it may be inefficient when subproblems overlap.

Part E: Example 1 — Climbing Stairs

11. The Climbing Stairs Problem

Problem:

> You are climbing stairs. Each move can be 1 step or 2 steps. How many different ways can you reach step `n`?

Examples:

```
n = 1
Ways:
1
Answer: 1
```

```
n = 2
Ways:
1 + 1
```

```
2
Answer: 2
```

```
n = 3
Ways:
1 + 1 + 1
1 + 2
2 + 1
Answer: 3
```

The pattern is:

```
climbStairs(n) = climbStairs(n - 1) + climbStairs(n - 2)
```

Why?

To reach step `n`, the final move must be either:

- 1 step from `n - 1`
- 2 steps from `n - 2`

So the total ways are:

```
ways to reach n - 1 + ways to reach n - 2
```

---

12. Climbing Stairs Base Cases

Base cases stop recursion.

For climbing stairs:

```
n = 1 -> 1 way
n = 2 -> 2 ways
```

So the base case can be:

```
if (n <= 2) {
  return n;
}
```

Simple meaning:

> If the answer is already obvious, return it immediately.

---

13. Climbing Stairs With Naive Recursion

```
function climbStairsRecursive(n) {
  if (n <= 2) {
    return n;
  }

  return climbStairsRecursive(n - 1) + climbStairsRecursive(n - 2);
}

console.log(climbStairsRecursive(5)); // 8
```

How to read it:

```
function climbStairsRecursive(n) {
```

Create a function that receives a stair number.

```
if (n <= 2) {
  return n;
}
```

If `n` is 1 or 2, return the known answer.

```
return climbStairsRecursive(n - 1) + climbStairsRecursive(n - 2);
```

Otherwise, build the answer from the two smaller stair problems.

---

14. Dry Run: Naive Recursion for `n = 5`

Call:

```
climbStairsRecursive(5)
```

Breaks into:

```
climbStairsRecursive(4) + climbStairsRecursive(3)
```

Then:

```
climbStairsRecursive(4)
```

breaks into:

```
climbStairsRecursive(3) + climbStairsRecursive(2)
```

Then each `climbStairsRecursive(3)` breaks into:

```
climbStairsRecursive(2) + climbStairsRecursive(1)
```

Repeated work:

```
climbStairsRecursive(3)
climbStairsRecursive(2)
climbStairsRecursive(1)
```

These values may be calculated again and again.

Result:

```
8
```

The answer is correct, but the method is inefficient.

Part F: Memoization

15. What Is Memoization?

Memoization means:

> Save answers from recursive calls so you can reuse them later.

Memoization usually uses:

- an object
- a Map
- an array

In beginner JavaScript, an object is common:

```
const memo = {};
```

Simple meaning:

> A memo is a storage place for answers you already calculated.

---

16. Memoization Is Top-Down DP

Memoization is called **top-down** dynamic programming.

Top-down means:

1. Start with the big problem.
2. Break it into smaller problems recursively.
3. Save answers when you calculate them.
4. Reuse saved answers later.

Simple meaning:

> Start from the question you were asked, then work downward into smaller questions.

---

17. Climbing Stairs With Memoization

```
function climbStairsMemo(n, memo = {}) {
  if (memo[n] !== undefined) {
```

```
    return memo[n];
  }

  if (n <= 2) {
    return n;
  }

  memo[n] = climbStairsMemo(n - 1, memo) + climbStairsMemo(n - 2, memo);

  return memo[n];
}

console.log(climbStairsMemo(5)); // 8
```

---

18. How the Memoized Code Works

```
function climbStairsMemo(n, memo = {}) {
```

The function receives:

- `n` : the stair number
- `memo` : an object that stores solved answers

```
if (memo[n] !== undefined) {
  return memo[n];
}
```

If the answer is already saved, return it immediately.

```
if (n <= 2) {
  return n;
}
```

If `n` is 1 or 2, return the known answer.

```
memo[n] = climbStairsMemo(n - 1, memo) + climbStairsMemo(n - 2, memo);
```

Calculate the answer and save it.

```
return memo[n];
```

Return the saved answer.

---

19. Dry Run: Memoization for `n = 5`

Start:

```
memo = {}
```

Call:

```
climbStairsMemo(5)
```

The function needs:

```
climbStairsMemo(4)
climbStairsMemo(3)
```

After solving smaller values, the memo may contain:

```
{
  3: 3,
  4: 5,
  5: 8
}
```

Now, if the function needs `climbStairsMemo(3)` again, it does not recalculate it.

It returns:

```
memo[3]
```

which is:

```
3
```

Simple meaning:

> Memoization turns repeated calculations into quick lookups.

---

20. Why `memo[n] !== undefined`?

This checks whether the answer already exists.

Example:

```
if (memo[n] !== undefined) {
  return memo[n];
}
```

Meaning:

> If `memo` already has an answer for `n`, return it.

Beginner note:

This works for climbing stairs because valid answers are numbers like `1`, `2`, `3`, `5`, `8`, and none of them are `undefined`.

---

21. Common Memoization Mistake

A common mistake is calculating the value but not saving it.

Weak version:

```
function climbStairsMemo(n, memo = {}) {
  if (n <= 2) {
    return n;
  }

  return climbStairsMemo(n - 1, memo) + climbStairsMemo(n - 2, memo);
}
```

This still repeats work.

Better version:

```
memo[n] = climbStairsMemo(n - 1, memo) + climbStairsMemo(n - 2, memo);
return memo[n];
```

Beginner rule:

> If you use memoization, remember to store the result.

Part G: Tabulation

22. What Is Tabulation?

Tabulation means:

> Build a table of answers from the smallest problem up to the final problem.

Tabulation usually uses an array:

```
const dp = new Array(n + 1).fill(0);
```

Simple meaning:

> Fill a table step by step until you reach the answer.

---

23. Tabulation Is Bottom-Up DP

Tabulation is called **bottom-up** dynamic programming.

Bottom-up means:

1. Start with the smallest known answers.
2. Use those answers to build bigger answers.
3. Continue until you reach the final answer.

Simple meaning:

> Start from the bottom and build upward.

---

24. Climbing Stairs With Tabulation

```
function climbStairsTable(n) {
  if (n <= 2) {
    return n;
  }

  const dp = new Array(n + 1).fill(0);

  dp[1] = 1;
  dp[2] = 2;

  for (let i = 3; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
  }

  return dp[n];
}

console.log(climbStairsTable(5)); // 8
```

---

25. How the Tabulation Code Works

```
if (n <= 2) {
  return n;
}
```

If the answer is already known, return it.

```
const dp = new Array(n + 1).fill(0);
```

Create a table large enough to store answers from `0` to `n`.

Why `n + 1`?

Because array indexes start at `0`.

If `n` is `5`, we need indexes:

```
0, 1, 2, 3, 4, 5
```

That is 6 slots.

```
dp[1] = 1;
dp[2] = 2;
```

Set the known base answers.

```
for (let i = 3; i <= n; i++) {
  dp[i] = dp[i - 1] + dp[i - 2];
}
```

Build each answer from the previous two answers.

```
return dp[n];
```

Return the final answer.

---

26. Dry Run: Tabulation for `n = 5`

Call:

```
climbStairsTable(5)
```

Create the table:

```
dp = [0, 0, 0, 0, 0, 0]
```

Set base cases:

```
dp[1] = 1;
dp[2] = 2;
```

Now:

```
dp = [0, 1, 2, 0, 0, 0]
```

Loop starts at `i = 3`.

Step 1

```
i = 3
```

```
dp[3] = dp[2] + dp[1]
dp[3] = 2 + 1
dp[3] = 3
```

Now:

```
dp = [0, 1, 2, 3, 0, 0]
```

Step 2

```
i = 4
```

```
dp[4] = dp[3] + dp[2]
dp[4] = 3 + 2
dp[4] = 5
```

Now:

```
dp = [0, 1, 2, 3, 5, 0]
```

Step 3

```
i = 5
```

```
dp[5] = dp[4] + dp[3]
dp[5] = 5 + 3
dp[5] = 8
```

Final table:

```
dp = [0, 1, 2, 3, 5, 8]
```

Answer:

```
8
```

Part H: Memoization vs Tabulation

27. Main Difference

Memoization and tabulation both save answers.

But they build the solution differently.

Feature	Memoization	Tabulation
Direction	Top-down	Bottom-up
Uses recursion?	Yes	Usually no
Storage	Object, Map, or array	Usually array
Starts with	Big problem	Smallest known answers
Good when	Recursive idea is easy	Iterative pattern is clear

Simple meaning:

> Memoization starts from the final question and works down. Tabulation starts from small answers and works up.

---

28. When to Use Memoization

Use memoization when:

- the recursive solution is easy to write
- you can clearly define base cases
- the same recursive calls repeat
- you want to improve recursion without fully rewriting it as a loop

Example pattern:

```
function solve(n, memo = {}) {
  if (memo[n] !== undefined) {
    return memo[n];
  }

  if (baseCase) {
    return baseAnswer;
  }

  memo[n] = solve(smallerInput, memo);
  return memo[n];
}
```

---

29. When to Use Tabulation

Use tabulation when:

- you know the smallest answers
- you can build bigger answers in order
- you want to avoid recursion
- you want a predictable loop-based solution

Example pattern:

```
function solve(n) {
  const dp = new Array(n + 1).fill(0);

  dp[0] = baseAnswer;

  for (let i = 1; i <= n; i++) {
    dp[i] = answerBuiltFromEarlierValues;
  }

  return dp[n];
}
```

Part I: Time and Space Complexity

30. Time Complexity

Time complexity means:

> How much work the algorithm does as the input grows.

For naive climbing stairs recursion, the time can grow very quickly because the function repeats many calls.

For memoization:

```
Each important value is calculated once.
```

For climbing stairs, that means values from `1` to `n`.

So the time complexity is:

```
O(n)
```

For tabulation, the loop also runs from `3` to `n`.

So the time complexity is also:

```
O(n)
```

Simple meaning:

> DP often improves repeated recursion into a solution that grows more directly with the input size.

---

31. Space Complexity

Space complexity means:

> How much extra memory the algorithm uses.

Memoization stores answers in a memo object.

For climbing stairs, it stores up to `n` answers.

Space:

```
O(n)
```

Tabulation stores a `dp` array with `n + 1` slots.

Space:

```
O(n)
```

But sometimes tabulation can be optimized.

Part J: Space Optimization

32. What Is Space Optimization?

Space optimization means:

> Use less memory while still getting the correct answer.

In climbing stairs, the tabulation version stores the whole array:

```
[0, 1, 2, 3, 5, 8]
```

But to calculate the next value, you only need the previous two values.

For example:

```
dp[5] = dp[4] + dp[3]
```

You do not need `dp[1]` anymore at that point.

Simple meaning:

> If you only need the last two answers, do not store every answer.

---

33. Climbing Stairs With Space Optimization

```
function climbStairsOptimized(n) {
  if (n <= 2) {
    return n;
  }

  let twoStepsBefore = 1;
```

```
  let oneStepBefore = 2;

  for (let i = 3; i <= n; i++) {
    const current = oneStepBefore + twoStepsBefore;
    twoStepsBefore = oneStepBefore;
    oneStepBefore = current;
  }

  return oneStepBefore;
}

console.log(climbStairsOptimized(5)); // 8
```

---

34. How the Optimized Code Works

```
let twoStepsBefore = 1;
let oneStepBefore = 2;
```

These represent:

```
ways to reach step 1 = 1
ways to reach step 2 = 2
```

Then:

```
const current = oneStepBefore + twoStepsBefore;
```

Calculate the current stair answer.

Then move the values forward:

```
twoStepsBefore = oneStepBefore;
oneStepBefore = current;
```

Simple meaning:

> Keep only the last two useful answers.

---

35. Dry Run: Optimized Climbing Stairs for `n = 5`

Start:

```
twoStepsBefore = 1
oneStepBefore = 2
```

Step 3

```
i = 3
current = 2 + 1 = 3
```

Update:

```
twoStepsBefore = 2
oneStepBefore = 3
```

Step 4

```
i = 4
current = 3 + 2 = 5
```

Update:

```
twoStepsBefore = 3
oneStepBefore = 5
```

Step 5

```
i = 5
current = 5 + 3 = 8
```

Update:

```
twoStepsBefore = 5
oneStepBefore = 8
```

Return:

```
8
```

Complexity:

```
Time: O(n)
Space: O(1)
```

Part K: Example 2 — Coin Change

36. The Coin Change Problem

Problem:

> Given a list of coin values and a target amount, find the minimum number of coins needed to make that amount.

Example:

```
coins = [1, 3, 4]
amount = 6
```

Possible ways:

```
1 + 1 + 1 + 1 + 1 + 1 = 6 coins
3 + 3 = 2 coins
4 + 1 + 1 = 3 coins
```

Best answer:

```
2 coins
```

because:

```
3 + 3 = 6
```

Simple meaning:

> We want the fewest coins, not just any coin combination.

---

37. Why Coin Change Can Use DP

Coin change has overlapping subproblems.

Example:

To solve amount `6`, you may need to know the best answer for:

```
6 - 1 = 5
6 - 3 = 3
6 - 4 = 2
```

So amount `6` depends on smaller amounts.

The same smaller amounts can appear again while solving other amounts.

Coin change also has optimal substructure.

The best answer for an amount can be built from the best answer for a smaller amount plus one coin.

Pattern:

```
dp[currentAmount] = Math.min(
  dp[currentAmount],
  dp[currentAmount - coin] + 1
);
```

Simple meaning:

> Try using each coin. Choose the option that uses the fewest coins.

---

38. Coin Change With Tabulation

```
function minCoins(amount, coins) {
  const dp = new Array(amount + 1).fill(Infinity);

  dp[0] = 0;
```

```
  for (let currentAmount = 1; currentAmount <= amount; currentAmount++) {
    for (const coin of coins) {
      if (coin <= currentAmount) {
        dp[currentAmount] = Math.min(
          dp[currentAmount],
          dp[currentAmount - coin] + 1
        );
      }
    }
  }

  if (dp[amount] === Infinity) {
    return -1;
  }

  return dp[amount];
}

console.log(minCoins(6, [1, 3, 4])); // 2
```

---

39. How the Coin Change Code Works

```
const dp = new Array(amount + 1).fill(Infinity);
```

Create an array to store the minimum coins needed for every amount from `0` to `amount`.

Example for amount `6`:

```
[Infinity, Infinity, Infinity, Infinity, Infinity, Infinity, Infinity]
```

There are 7 positions because we need indexes:

```
0, 1, 2, 3, 4, 5, 6
```

---

40. Why Use `Infinity`?

`Infinity` means:

> We do not know a valid answer yet.

At the start, every amount is treated as impossible.

Then the algorithm improves the values when it finds valid coin combinations.

Example:

```
dp = [0, Infinity, Infinity, Infinity, Infinity, Infinity, Infinity]
```

Simple meaning:

> Start with impossible values, then replace them with better answers.

---

41. Why `dp[0] = 0`?

```
dp[0] = 0;
```

This means:

> It takes 0 coins to make amount 0.

This is the base case.

Without this base case, the algorithm has no starting point.

Example:

To make amount `1` with coin `1`:

```
dp[1] = dp[1 - 1] + 1
dp[1] = dp[0] + 1
dp[1] = 0 + 1
dp[1] = 1
```

That works because `dp[0]` is known.

---

42. The Outer Loop

```
for (let currentAmount = 1; currentAmount <= amount; currentAmount++) {
```

This loop goes through every amount from `1` to the target amount.

For amount `6`, it checks:

```
1, 2, 3, 4, 5, 6
```

Simple meaning:

> Solve every smaller amount before solving the final amount.

---

43. The Inner Loop

```
for (const coin of coins) {
```

This loop tries every coin.

For coins:

```
[1, 3, 4]
```

It tries:

```
coin = 1
coin = 3
coin = 4
```

Simple meaning:

> For each amount, try every coin and keep the best result.

---

44. Why Check `coin <= currentAmount`?

```
if (coin <= currentAmount) {
```

You can only use a coin if it is not bigger than the current amount.

Example:

If current amount is `2`, you cannot use coin `3` or coin `4`.

Valid:

```
coin 1 for amount 2
```

Invalid:

```
coin 3 for amount 2
coin 4 for amount 2
```

Simple meaning:

> Do not use a coin that is too large for the current amount.

---

45. The Main Coin Change Formula

```
dp[currentAmount] = Math.min(
  dp[currentAmount],
  dp[currentAmount - coin] + 1
);
```

This means:

> Keep the better option: the old answer or the new answer using this coin.

Break it down:

```
dp[currentAmount]
```

The best answer we currently know.

```
dp[currentAmount - coin] + 1
```

The answer if we use this coin.

Why `+ 1`?

Because we are adding one coin.

Example:

If:

```
currentAmount = 6
coin = 3
```

Then:

```
dp[6 - 3] + 1
dp[3] + 1
```

If `dp[3]` is `1`, then:

```
1 + 1 = 2
```

So amount `6` can be made with 2 coins:

```
3 + 3
```

---

46. Why Return `-1` If Still `Infinity`?

```
if (dp[amount] === Infinity) {
  return -1;
}
```

If the final answer is still `Infinity`, no coin combination worked.

So the function returns `-1`.

Simple meaning:

> Return `-1` when the target amount cannot be made.

Example:

```
console.log(minCoins(3, [2])); // -1
```

You cannot make amount `3` using only coin `2`.

---

47. Coin Change Dry Run for `coins = [1, 3, 4]` and `amount = 6`

Start:

```
dp = [0, Infinity, Infinity, Infinity, Infinity, Infinity, Infinity]
```

Amount 1

Try coin `1`:

```
dp[1] = dp[0] + 1 = 1
```

Now:

```
dp = [0, 1, Infinity, Infinity, Infinity, Infinity, Infinity]
```

Amount 2

Try coin `1`:

```
dp[2] = dp[1] + 1 = 2
```

Now:

```
dp = [0, 1, 2, Infinity, Infinity, Infinity, Infinity]
```

Amount 3

Try coin `1`:

```
dp[3] = dp[2] + 1 = 3
```

Try coin `3`:

```
dp[3] = dp[0] + 1 = 1
```

Best is `1`.

Now:

```
dp = [0, 1, 2, 1, Infinity, Infinity, Infinity]
```

Amount 4

Try coin `1`:

```
dp[4] = dp[3] + 1 = 2
```

Try coin `3`:

```
dp[4] = dp[1] + 1 = 2
```

Try coin `4`:

```
dp[4] = dp[0] + 1 = 1
```

Best is `1`.

Now:

```
dp = [0, 1, 2, 1, 1, Infinity, Infinity]
```

Amount 5

Try coin `1`:

```
dp[5] = dp[4] + 1 = 2
```

Try coin `3`:

```
dp[5] = dp[2] + 1 = 3
```

Try coin `4`:

```
dp[5] = dp[1] + 1 = 2
```

Best is `2`.

Now:

```
dp = [0, 1, 2, 1, 1, 2, Infinity]
```

Amount 6

Try coin `1`:

```
dp[6] = dp[5] + 1 = 3
```

Try coin `3`:

```
dp[6] = dp[3] + 1 = 2
```

Try coin `4`:

```
dp[6] = dp[2] + 1 = 3
```

Best is `2`.

Final table:

```
dp = [0, 1, 2, 1, 1, 2, 2]
```

Answer:

```
2
```

Meaning:

```
3 + 3 = 6
```

Part L: When to Use Dynamic Programming

48. Signs That DP May Be Useful

Use dynamic programming when you see these signs:

- The same recursive calls repeat.
- The problem asks for the number of ways.
- The problem asks for a minimum value.
- The problem asks for a maximum value.
- The problem asks for the best possible result.
- The final answer can be built from smaller answers.
- A brute-force solution checks too many combinations.

Simple rule:

> If smaller answers repeat and help build the final answer, think about DP.

---

49. Common DP Problem Types

Problem Type	Example Question
Counting ways	How many ways can you climb stairs?
Minimum cost	What is the minimum number of coins?
Maximum value	What is the most value you can carry?
Path decisions	What is the cheapest path?
Text processing	How many edits are needed to change one word into another?
Scheduling	What is the best use of limited time?
Resource allocation	How should limited resources be divided?

---

50. Common Real-World DP Use Cases

Dynamic programming can appear in problems related to:

- route optimization
- text comparison
- spell-checking ideas
- budgeting
- scheduling
- financial modeling
- resource allocation
- game scoring
- path finding
- knapsack-style problems

Beginner note:

You do not need to master all of these now. The main goal is to recognize the pattern.

Part M: Common Beginner Mistakes

51. Mistake 1: Using DP When There Is No Repeated Work

Not every problem needs DP.

If there are no repeated subproblems, DP may make the solution more complicated than necessary.

Weak habit:

> Use DP for every algorithm problem.

Better habit:

> Use DP only when repeated subproblems exist and saving answers helps.

---

52. Mistake 2: Forgetting the Base Case

Recursive DP needs base cases.

Without a base case, recursion may never stop.

Bad example:

```
function climb(n) {
  return climb(n - 1) + climb(n - 2);
}
```

This has no stopping condition.

Better:

```
function climb(n) {
  if (n <= 2) {
    return n;
  }

  return climb(n - 1) + climb(n - 2);
}
```

Beginner rule:

> Every recursive solution needs a stopping point.

---

53. Mistake 3: Forgetting to Store the Memoized Answer

Bad:

```
return solve(n - 1, memo) + solve(n - 2, memo);
```

Better:

```
memo[n] = solve(n - 1, memo) + solve(n - 2, memo);
return memo[n];
```

Beginner rule:

> Memoization only helps if you save the answer.

---

54. Mistake 4: Creating the DP Array With the Wrong Size

For a target `n`, you often need indexes from `0` through `n`.

That means the array size should often be:

```
n + 1
```

Example:

```
const dp = new Array(n + 1).fill(0);
```

If `n` is `5`, you need:

```
0, 1, 2, 3, 4, 5
```

That is 6 slots.

---

55. Mistake 5: Confusing Indexes With Values

In a DP array, the index often represents a subproblem.

Example:

```
dp[5]
```

In climbing stairs, this means:

```
number of ways to reach step 5
```

In coin change, this means:

```
minimum coins needed to make amount 5
```

Beginner rule:

> Always define what `dp[i]` means before writing the loop.

---

56. Mistake 6: Not Understanding the DP Formula

Do not memorize formulas blindly.

For climbing stairs:

```
dp[i] = dp[i - 1] + dp[i - 2]
```

Meaning:

> Ways to reach this step = ways from one step before + ways from two steps before.

For coin change:

```
dp[currentAmount] = Math.min(
  dp[currentAmount],
  dp[currentAmount - coin] + 1
);
```

Meaning:

> Keep the fewest coins possible.

Beginner rule:

> A DP formula should match the story of the problem.

---

57. Mistake 7: Returning the Wrong Value

In tabulation, the final answer is often:

```
dp[n]
```

or:

```
dp[amount]
```

Not always the whole array.

Example:

```
return dp[n];
```

Simple meaning:

> Return the final subproblem answer, not the entire table unless you are debugging.

Part N: DP Problem-Solving Workflow

58. Step-by-Step DP Workflow

When solving a DP problem, use this process:

1. Understand the problem.
2. Write small examples.
3. Identify what the function should return.
4. Look for repeated subproblems.
5. Check if smaller answers build bigger answers.
6. Define the base cases.
7. Define what `dp[i]` or `memo[n]` means.
8. Write the recurrence formula.
9. Choose memoization or tabulation.
10. Write the code.
11. Dry-run a small input.
12. Analyze time and space complexity.
13. Optimize space if possible.

---

59. Questions to Ask Before Coding

Ask:

- What is the input?
- What is the output?
- What is the smallest input?
- What are the base cases?
- What smaller problems does this problem depend on?
- Do smaller problems repeat?
- What should I store?
- Should I use memoization or tabulation?
- What does each table entry mean?
- What should I return?

Part O: Important DP Patterns

60. Memoization Pattern

```
function solve(n, memo = {}) {
  if (memo[n] !== undefined) {
    return memo[n];
  }

  if (baseCase) {
    return baseAnswer;
  }

  memo[n] = solve(smallerProblem, memo);
  return memo[n];
}
```

Use this when recursive thinking is natural.

---

61. Tabulation Pattern

```
function solve(n) {
  const dp = new Array(n + 1).fill(0);

  dp[0] = baseAnswer;

  for (let i = 1; i <= n; i++) {
    dp[i] = answerBuiltFromEarlierValues;
  }

  return dp[n];
}
```

Use this when you can build answers from small to large.

---

62. Space-Optimized Pattern

```
function solve(n) {
  let previous = baseAnswer1;
  let current = baseAnswer2;
```

```
  for (let i = 3; i <= n; i++) {
    const next = previous + current;
    previous = current;
    current = next;
  }

  return current;
}
```

Use this when you only need a few previous values.

Part P: Practice Problems

63. Practice 1: Identify DP Conditions

For each problem, ask:

1. Are there overlapping subproblems?
2. Is there optimal substructure?

Problems:

```
A. Count ways to climb stairs.
B. Find the first item in an array.
C. Find minimum coins for an amount.
D. Reverse a string.
```

Expected thinking:

Problem	DP Useful?	Why?
Count ways to climb stairs	Yes	Smaller stair answers repeat.
Find first item	No	Just return `arr[0]`.
Minimum coins	Yes	Smaller amount answers build bigger amount answers.
Reverse a string	Usually no	No repeated subproblems needed.

---

64. Practice 2: Complete the Memoized Function

Fill in the missing line:

```
function climbStairsMemo(n, memo = {}) {
  if (memo[n] !== undefined) {
    return memo[n];
  }

  if (n <= 2) {
    return n;
  }

  // Missing line here

  return memo[n];
}
```

Correct answer:

```
memo[n] = climbStairsMemo(n - 1, memo) + climbStairsMemo(n - 2, memo);
```

---

65. Practice 3: Complete the Tabulation Function

Fill in the missing line:

```
function climbStairsTable(n) {
  if (n <= 2) {
    return n;
  }

  const dp = new Array(n + 1).fill(0);
  dp[1] = 1;
  dp[2] = 2;

  for (let i = 3; i <= n; i++) {
    // Missing line here
  }

  return dp[n];
}
```

Correct answer:

```
dp[i] = dp[i - 1] + dp[i - 2];
```

---

66. Practice 4: Dry Run the DP Table

For:

```
climbStairsTable(6)
```

Fill the table:

```
dp[1] = 1
dp[2] = 2
dp[3] = ?
dp[4] = ?
dp[5] = ?
dp[6] = ?
```

Answer:

```
dp[1] = 1
dp[2] = 2
dp[3] = 3
dp[4] = 5
dp[5] = 8
dp[6] = 13
```

---

67. Practice 5: Coin Change

Predict the answer:

```
console.log(minCoins(6, [1, 3, 4]));
```

Answer:

```
2
```

Reason:

```
3 + 3 = 6
```

Part Q: Sprint 25 Mastery Checklist

You understand Sprint 25 when you can do these:

- Explain dynamic programming in simple words.
- Explain overlapping subproblems.
- Explain optimal substructure.
- Explain why naive recursion can be slow.
- Write a naive recursive climbing stairs solution.
- Write a memoized climbing stairs solution.
- Write a tabulated climbing stairs solution.
- Write a space-optimized climbing stairs solution.
- Explain what a `memo` object stores.
- Explain what a `dp` array stores.
- Dry-run a memoized solution.
- Dry-run a tabulated solution.
- Explain why `dp[0] = 0` matters in coin change.
- Explain why `Infinity` is used in coin change.
- Write the coin change tabulation solution.
- Explain time complexity for memoization and tabulation.
- Explain space complexity for memoization and tabulation.
- Recognize when DP is useful.
- Recognize when DP is not necessary.

Part R: Key Vocabulary

Dynamic Programming

A technique that saves smaller answers and reuses them.

DP

Short for dynamic programming.

Subproblem

A smaller version of the main problem.

Overlapping Subproblems

The same smaller problems appear again and again.

Optimal Substructure

The big answer can be built from smaller answers.

Naive Recursion

A simple recursive solution that does not save repeated answers.

Memoization

Saving recursive results so repeated calls are faster.

Top-Down

Start with the big problem and recursively move to smaller problems.

Tabulation

Building a table of answers from small problems to big problems.

Bottom-Up

Start with the smallest answers and build up to the final answer.

DP Array

An array that stores answers to subproblems.

Memo Object

An object that stores answers already calculated by recursion.

Base Case

A known answer that stops recursion or starts a DP table.

Space Optimization

Reducing memory usage by storing only the values you still need.

Time Complexity

How much work an algorithm does as input grows.

Space Complexity

How much extra memory an algorithm uses as input grows.

Part S: Final Sprint 25 Summary

Sprint 25 is about making repeated algorithm work faster.

Naive recursion says:

```
Solve the same smaller problems again and again.
```

Dynamic programming says:

```
Save smaller answers and reuse them.
```

The two major DP approaches are:

```
Memoization = top-down + recursion + saved answers
Tabulation = bottom-up + loops + table of answers
```

The two main signs of a DP problem are:

```
Overlapping subproblems
Optimal substructure
```

For climbing stairs:

```
dp[i] = dp[i - 1] + dp[i - 2];
```

For coin change:

```
dp[currentAmount] = Math.min(
  dp[currentAmount],
  dp[currentAmount - coin] + 1
);
```

Core idea:

> Dynamic programming trades memory for speed. It uses storage to avoid repeated calculations.

Final beginner rule:

> Before using DP, ask: “Am I solving the same smaller problem more than once?”